

UNIVERSIDADE ESTADUAL DA PARAÍBA
CAMPUS I – EDVALDO DE SOUZA DO Ó
CENTRO DE CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE CIÊNCIA DA COMPUTAÇÃO

SUELLE FERREIRA MACIEL

COORDENAÇÃO POR ESPERA OCUPADA

Relatório 2 da atividade 4 de Sistemas Operacionais

CAMPINA GRANDE
2026

1 INTRODUÇÃO

Esta atividade analisa os problemas causados pelo acesso concorrente de duas threads POSIX a uma estrutura de dados compartilhada e compara soluções de coordenação baseadas em espera ocupada. As duas threads manipulam uma fila circular de 10 inteiros: em cada iteração, uma thread retira o primeiro elemento, sorteia um novo valor entre 0 e 99, insere-o no fim da fila e imprime a operação e o estado da fila.

Foram implementadas três versões do programa: sem mecanismo de sincronização, com alternância estrita e com o algoritmo de Peterson. Em seguida, mediu-se o desempenho de cada versão em inserções por segundo. A espera ocupada consiste em fazer a tarefa testar continuamente uma condição até que possa entrar na seção crítica, o trecho de código que acessa os dados compartilhados (MAZIERO, 2019).

2 AMBIENTE DE EXECUÇÃO

A atividade foi desenvolvida em um computador com Windows, por meio do WSL2 (Windows Subsystem for Linux) com a distribuição Ubuntu 24.04, que executa um núcleo Linux real e oferece suporte completo às threads POSIX (LINUX MAN-PAGES PROJECT, 2026). A preparação e a execução seguiram as etapas abaixo.

1. **Instalação do WSL e do Ubuntu**, por meio do PowerShell executado como administrador, conforme a documentação oficial (MICROSOFT, 2025):

```
wsl --install -d Ubuntu-24.04
```

2. **Instalação do compilador e do make**, no terminal do Ubuntu:

```
sudo apt update && sudo apt install build-essential
```

3. **Acesso à pasta do projeto**, cujo caminho no WSL começa em /mnt/c, correspondente ao disco C: do Windows:

```
cd /mnt/c/Users/suell/Documents/UEPB/2026.2/
  SistemasOperacionais/Atividade04/4.2
```

4. **Compilação** das três versões com o arquivo Makefile, que usa as opções -O2 (otimização) e -pthread (suporte a threads):

```
make
```

5. **Observação do comportamento da fila.** Para obter capturas de tamanho controlado, a saída foi limitada às 20 primeiras linhas com head; na versão sem sincronização, o comando grep filtrou as 10 primeiras operações em que a fila ficou inconsistente:

```
./sem_sincronizacao | grep -m 10 INCONSISTENTE
./alternancia | head -20
./peterson | head -20
```

6. **Medição de desempenho**, com o modo -m, que executa o programa durante o número de segundos indicado, sem imprimir as operações:

```
./sem_sincronizacao -m 10
./alternancia -m 10
./peterson -m 10
```

3 IMPLEMENTAÇÃO

O código foi organizado de modo que as três versões compartilhem o mesmo programa principal e se diferenciem apenas pelo mecanismo de sincronização, conforme o Quadro 1.

Quadro 1 – Arquivos que compõem a solução

Arquivo	**Conteúdo**
fila.h e fila.c	Fila circular de inteiros com capacidade para 10 elementos e comportamento FIFO
comum.h e comum.c	Programa principal: inicializa a fila cheia, cria as duas threads e realiza a medição
sem_sincronizacao.c	Funções entra() e sai() vazias
alternancia.c	Funções entra() e sai() com alternância estrita
peterson.c	Funções entra() e sai() com o algoritmo de Peterson
Makefile	Compilação das três versões

Fonte: Elaborada pela autora (2026).

Cada thread executa um laço infinito em que chama entra(), retira o primeiro elemento, insere um novo, imprime a operação e chama sai(). Assim, toda a manipulação da fila e a impressão ocorrem dentro da seção crítica. Para sortear os

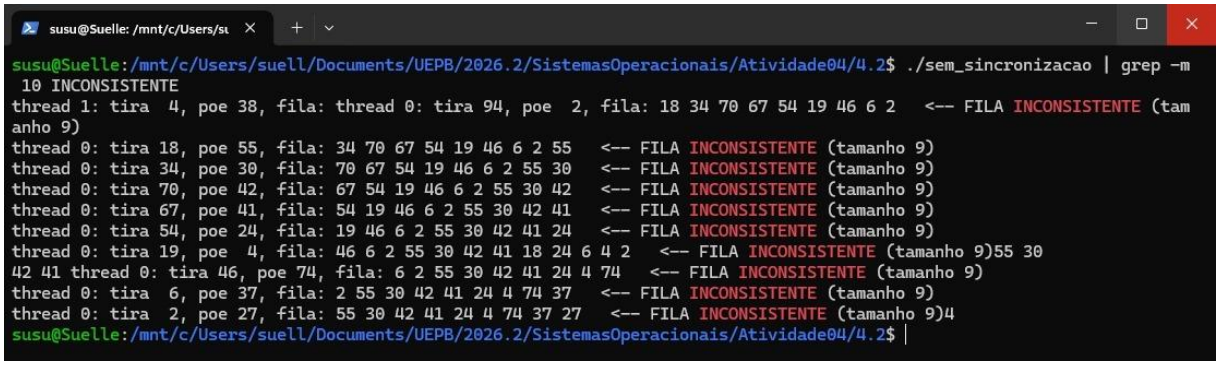
valores, utilizou-se rand_r(), com uma semente própria por thread, em vez de random(): na biblioteca C do Linux, random() usa uma trava interna, o que introduziria uma sincronização oculta entre as threads e distorceria a comparação.

Para facilitar a análise, o programa verifica a fila após cada operação. Se uma retirada ou inserção falhar, ou se o tamanho da fila for diferente de 10, a linha impressa recebe a marcação FILA INCONSISTENTE, com o tamanho encontrado.

4 COMPORTAMENTO DA FILA

4.1 SEM SINCRONIZAÇÃO (ITEM 1)

Figura 1 – Operações inconsistentes na versão sem sincronização

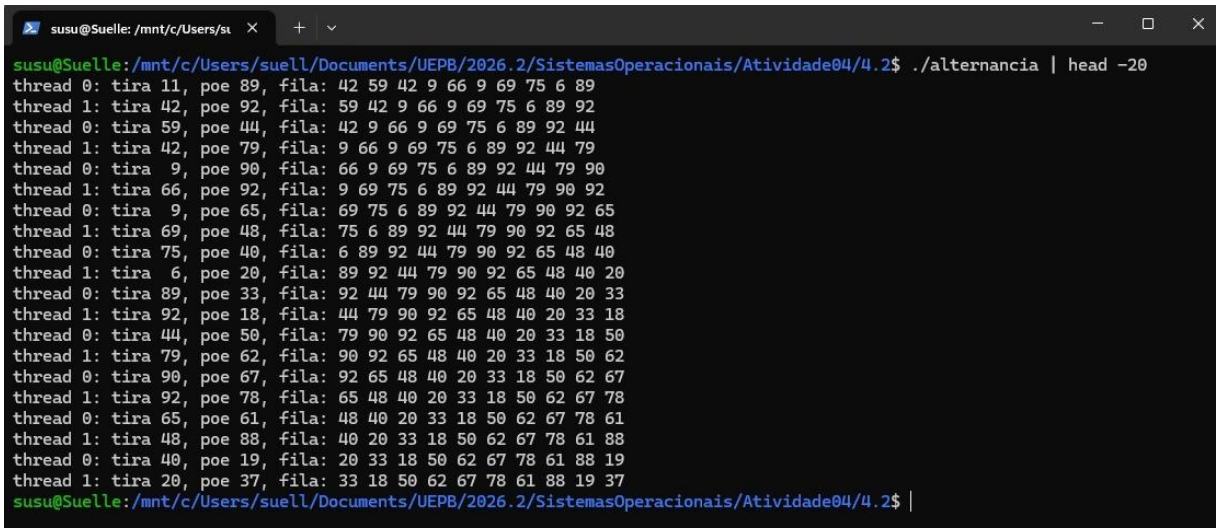


Fonte: Elaborada pela autora (2026).

A fila **não se comporta corretamente**. Como as duas threads alteram simultaneamente os campos inicio, fim e tamanho, a atualização feita por uma delas pode ser sobrescrita pela outra, o que caracteriza uma **condição de corrida** (MAZIERO, 2019). Na Figura 1, a fila passou a ter 9 elementos em vez de 10, o que indica que uma inserção se perdeu. As linhas impressas pelas duas threads também aparecem misturadas, pois a escrita na tela não é coordenada.

4.2 ALTERNÂNCIA ESTRITA (ITEM 2)

Figura 2 – Execução com alternância estrita

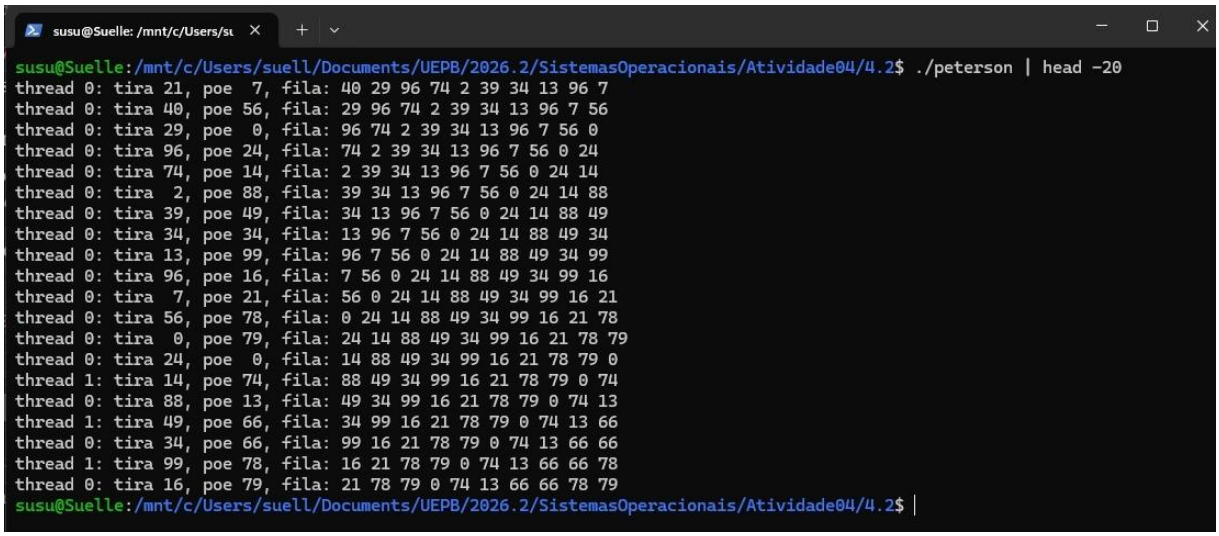


Fonte: Elaborada pela autora (2026).

A fila manteve sempre 10 elementos e comportamento FIFO, e as threads entraram na seção crítica exatamente na ordem t0, t1, t0, t1 (Figura 2). Esse comportamento decorre do próprio algoritmo: uma variável vez indica qual thread pode entrar, e cada thread, ao sair, passa a vez para a outra. A limitação é que nenhuma thread pode entrar duas vezes seguidas, mesmo que a outra não esteja interessada em entrar.

4.3 ALGORITMO DE PETERSON (ITEM 3)

Figura 3 – Execução com o algoritmo de Peterson



Fonte: Elaborada pela autora (2026).

A fila também se manteve correta, mas a ordem de entrada não é fixa (Figura 3). No início da execução, a thread 0 entrou 14 vezes seguidas, porque a thread 1 ainda não havia começado a disputar a seção crítica; a partir daí, as entradas passaram a se intercalar. Isso mostra a vantagem do algoritmo de Peterson sobre a alternância estrita: uma thread só espera quando a outra também quer entrar (PETERSON, 1981).

Na implementação, as variáveis de controle (quer[2] e vez) foram declaradas como atômicas, com o tipo atomic_int. Com variáveis comuns, o algoritmo pode falhar em processadores atuais, que podem reordenar uma escrita seguida de uma leitura na memória; nesse caso, as duas threads poderiam entrar ao mesmo tempo na seção crítica.

5 MEDIÇÃO DE DESEMPENHO

5.1 FORMA DE MEDIÇÃO (ITEM 4)

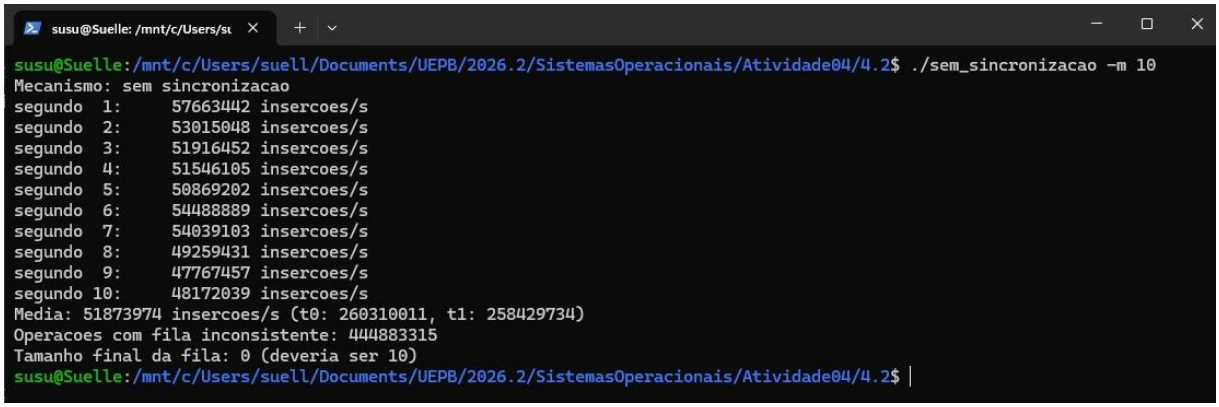
Para medir o número de inserções por segundo, cada thread incrementa um contador próprio a cada inserção bem-sucedida. Os contadores foram separados em linhas de cache distintas, para que o incremento feito por uma thread não interfira no da outra. Enquanto as duas threads trabalham, a thread principal do programa lê a

soma dos contadores a cada segundo, calcula a diferença em relação à leitura anterior e imprime a taxa. Ao final, exibe a média do período, o total de cada thread, o número de operações com fila inconsistente e o tamanho final da fila.

Durante a medição, as operações não são impressas na tela, pois a escrita no terminal é muito mais lenta que a manipulação da fila e dominaria o tempo medido. Cada versão foi executada por 10 segundos.

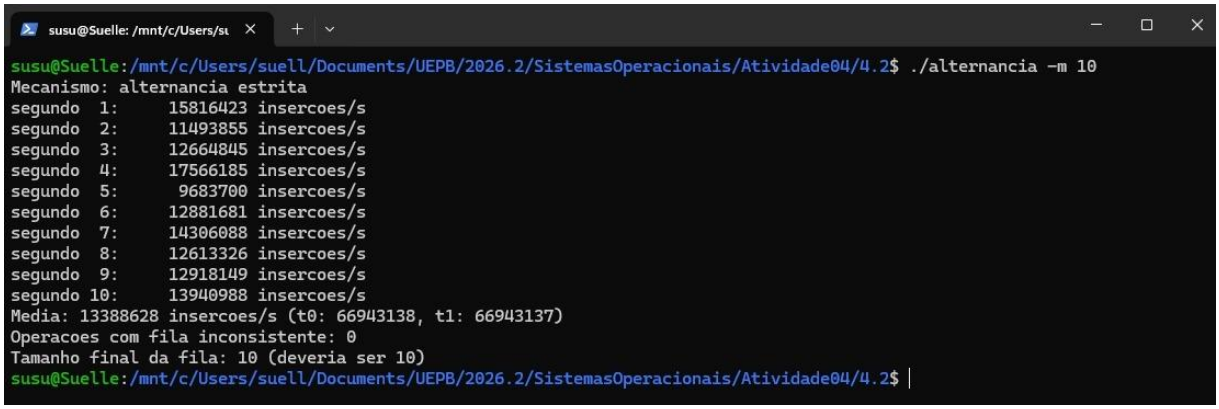
5.2 RESULTADOS (ITEM 5)

Figura 4 – Medição da versão sem sincronização



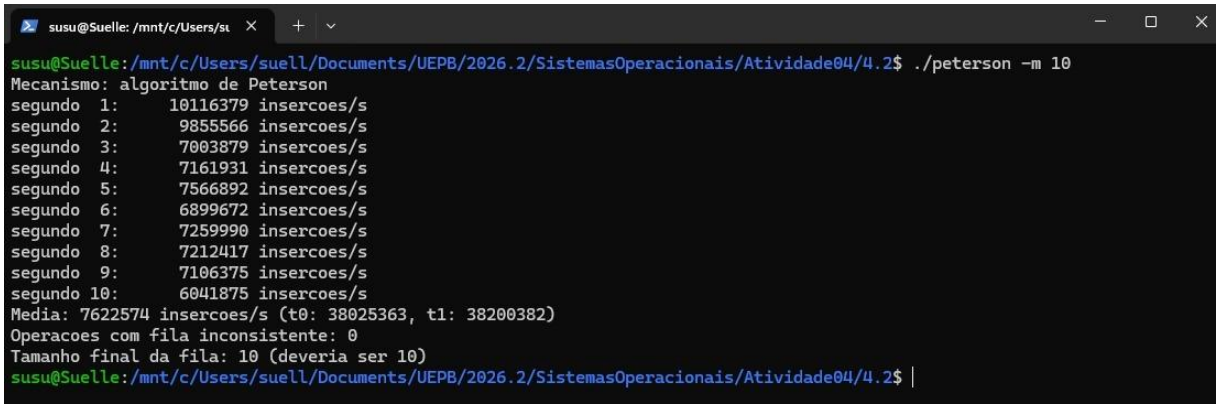
Fonte: Elaborada pela autora (2026).

Figura 5 – Medição da versão com alternância estrita



Fonte: Elaborada pela autora (2026).

Figura 6 – Medição da versão com o algoritmo de Peterson



Fonte: Elaborada pela autora (2026).

A Tabela 1 resume as medições das Figuras 4, 5 e 6.

Tabela 1 – Desempenho das três implementações em 10 segundos de execução

Versão	**Média (inserções/s)**	**Operações inconsistentes**	**Tamanho final da fila**
Sem sincronização	51.873.974	444.883.315	0
Alternância estrita	13.388.628	0	10
Algoritmo de Peterson	7.622.574	0	10

Fonte: Elaborada pela autora (2026).

5.3 ANÁLISE DOS RESULTADOS

A versão **sem sincronização** foi a mais rápida, com cerca de 3,9 vezes a taxa da alternância estrita e 6,8 vezes a do algoritmo de Peterson, porque as threads nunca esperam uma pela outra e executam de fato em paralelo. O resultado, porém, é incorreto: houve 444.883.315 operações com a fila inconsistente, e a fila terminou com 0 elementos em vez de 10. Desempenho sem correção não tem utilidade.

As duas versões com sincronização mantiveram a fila correta, mas foram mais lentas por dois motivos. Primeiro, na espera ocupada, a thread que aguarda ocupa um núcleo inteiro apenas testando uma variável, sem realizar trabalho útil (MAZIERO, 2019). Segundo, as variáveis de controle são compartilhadas entre os núcleos do processador e precisam ser transferidas de uma memória cache para a outra a cada troca de vez.

A **alternância estrita** foi cerca de 1,8 vez mais rápida que o **algoritmo de Peterson**. Como as duas threads querem entrar na seção crítica o tempo todo, a alternância funciona como uma passagem de vez perfeita: a cada entrada, cada thread lê a variável vez e, ao sair, faz uma única escrita. Os totais de cada thread confirmam essa divisão exata do trabalho: 66.943.138 inserções da thread 0 e 66.943.137 da thread 1. O algoritmo de Peterson, por sua vez, realiza duas escritas atômicas ao entrar (quer[id] e vez) e uma ao sair, além de ler duas variáveis na espera. Cada escrita atômica com ordem sequencial exige uma barreira de memória, que obriga o processador a concluir as operações pendentes antes de prosseguir, o que torna cada entrada mais custosa.

A vantagem do algoritmo de Peterson, que é permitir entradas consecutivas quando a outra thread não está interessada, não aparece neste experimento, porque as duas threads disputam a seção crítica continuamente. Em cenários em que uma das threads passasse parte do tempo fora da seção crítica, a alternância estrita obrigaria a outra a esperar sem necessidade, e o algoritmo de Peterson tenderia a ser mais eficiente.

Por fim, observou-se variação entre os segundos de uma mesma execução: no algoritmo de Peterson, por exemplo, a taxa variou de 6.041.875 a 10.116.379 inserções/s. Essa variação decorre de fatores externos ao programa, como o escalonamento das threads pelo sistema operacional e outros processos em execução no computador.

6 CONCLUSÃO

O experimento mostrou que o acesso concorrente sem coordenação a uma estrutura compartilhada produz resultados incorretos, mesmo em uma operação simples como retirar e inserir elementos em uma fila. A alternância estrita e o algoritmo de Peterson garantiram a exclusão mútua e mantiveram a fila consistente durante todas as execuções.

Por outro lado, a espera ocupada mostrou um custo elevado: as versões sincronizadas foram de 3,9 a 6,8 vezes mais lentas que a versão sem sincronização. Neste cenário de disputa contínua, a alternância estrita superou o algoritmo de Peterson, por executar menos operações atômicas por entrada. O algoritmo de Peterson, entretanto, é mais geral, pois não obriga as threads a se alternarem. Os

resultados também evidenciam que, nos processadores atuais, a correção desses algoritmos depende de variáveis atômicas, e não apenas da lógica do algoritmo.

REFERÊNCIAS

LINUX MAN-PAGES PROJECT. **pthreads(7)**: POSIX threads. Versão 6.19. [*S. l.*], 2026. Disponível em: https://man7.org/linux/man-pages/man7/pthreads.7.html. Acesso em: 15 out. 2026.

MAZIERO, Carlos Alberto. **Sistemas operacionais**: conceitos e mecanismos. Curitiba: Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2. Disponível em: https://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start. Acesso em: 15 out. 2026.

MICROSOFT. **Como instalar o Linux no Windows com o WSL**. [*S. l.*]: Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/pt-br/windows/wsl/install. Acesso em: 15 out. 2026.

PETERSON, Gary L. Myths about the mutual exclusion problem. **Information Processing Letters**, [*S. l.*], v. 12, n. 3, p. 115-116, 1981.